Tutoría de Recursividad en Java

https://gemini.google.com/app/8a07c01501ad1363

User prompt: Hola, mi nombre es Jaime Alfredo Pérez Rivera, de 4 grado y grupo A. Estoy en la materia de Estructura de Datos y necesito que actúes como mi tutor guiado. Quiero diseñar un método recursivo en Java para verificar si una palabra es palíndromo usando el patrón wrapper. No me des el código resuelto; hazme preguntas para guiarme en el análisis del caso base, el paso recursivo y la implementación.

Response: ¡Hola, Jaime Alfredo! Qué gusto saludarte. Un saludo especial para la clase de Estructura de Datos de 4.º A. Vamos a diseñar juntos ese algoritmo recursivo en Java usando el patrón *wrapper* (o método envoltorio) 💻.

Yo te iré guiando paso a paso con preguntas para que tú mismo construyas la solución.

1. Entendiendo la idea del palíndromo 🔄

Una palabra es un palíndromo si se lee igual de izquierda a derecha que de derecha a izquierda (por ejemplo, `"reconocer"`, `"oso"` o `"radar"`).

Para analizar esto recursivamente, solemos comparar el primer carácter con el último.

- **Pregunta de inicio:** Si comparas la primera y la última letra de la palabra `"reconocer"` y ves que son iguales, **¿qué subproblema más pequeño te queda por resolver para saber si el resto de la palabra también es un palíndromo?**

User prompt: Revisar sí las demás letras son iguales, una por una

Response: Exactamente 🎯. Al verificar que las letras de los extremos son iguales, nos queda por revisar la **subcadena interior** (es decir, la palabra sin el primer ni el último carácter).

Por ejemplo, si la palabra es `"reconocer"` y las 'r' de los extremos coinciden, el subproblema más pequeño es revisar `"econo"`.

Ahora pensemos en las condiciones de parada de nuestro método.

2. Identificando los Casos Base 🛑

La recursividad necesita saber cuándo detenerse para no ejecutarse infinitamente.

Imagínate que seguimos quitando letras de los extremos paso a paso:

1. **¿Qué pasa si la palabra tiene 1 sola letra (por ejemplo, `"o"`)?** ¿Es un palíndromo?
2. **¿Qué pasa si la palabra tiene 0 letras (cadena vacía `""`)?** ¿Es un palíndromo?
3. **¿Qué ocurre si en algún paso la primera letra y la última NO son iguales?**

Con base en esto, **¿cuáles serían los casos base de nuestro algoritmo y qué debería retornar cada uno (`true` o `false`)?**

User prompt: 1. Verdadero, 2. Falso, 3. Falso

Response: ¡Casi perfecto, Jaime! Tienes muy claras las ideas de la comparación 👏.

Hagamos una pequeña precisión en el caso 2:

- **Caso 1 (1 letra):** `true` ✅. Correcto, una palabra como `"o"` se lee igual al derecho y al revés.
- **Caso 3 (Letras distintas):** `false` ✅. Correcto, en el momento en que no coinciden los extremos, la palabra deja de ser un palíndromo.
- **Caso 2 (0 letras / cadena vacía `""`):** En lógica de programación, si llegamos a una cadena sin letras porque ya comparamos y quitamos todas las parejas con éxito, **se considera un palíndromo válido (`true`)** 💡.

Por lo tanto, resumimos que si la longitud de la cadena es **0 o 1** (es decir, `longitud <= 1`), el método debe retornar `true`.

3. El patrón *Wrapper* (Método Envoltorio) 🎁

Para no tener que estar recortando la cadena con `substring()` en cada llamada (lo cual consume mucha memoria), es más eficiente pasar la misma palabra y utilizar **dos índices de posición**: `inicio` y `fin`.

El usuario final solo quiere llamar a un método sencillo como: `esPalindromo("reconocer")`

Pero el método recursivo necesita más datos para trabajar: `esPalindromoRecursivo("reconocer", 0, 8)`

- **Pregunta:** ¿Por qué crees que el método *wrapper* (público) es útil aquí y qué función cumple al llamar al método recursivo (privado)?

User prompt: Es eficiente, ya que usa dos contadores los cuales inician al inicio y al final de la palabra, hasta chocar en la misma posición, y permite llamar los datos de manera más simple

Response: ¡Excelente deducción, Jaime! 🎯 Lo has explicado súper claro.

El método ***wrapper*** (público) le facilita la vida al usuario ocultando la complejidad: el usuario solo pasa la palabra, y el *wrapper* se encarga de iniciar el proceso pasando las posiciones iniciales (`inicio = 0` y `fin = palabra.length() - 1`) al método **recursivo privado**.

Además, como bien dijiste, mover esos dos índices (o contadores) hacia el centro hasta cruzar o chocar es mucho más eficiente en memoria que estar recortando la palabra.

4. Armando la lógica recursiva con índices 🧩

Pensemos cómo cambian los índices en cada paso:

- Empezamos con `inicio = 0` y `fin = palabra.length() - 1`.
- Verificamos que los caracteres en esas posiciones sean iguales: `palabra.charAt(inicio) == palabra.charAt(fin)`.

Si coinciden, debemos hacer la llamada recursiva para analizar el siguiente par de letras más interno.

Preguntas para definir el paso recursivo:

1. Para la siguiente llamada recursiva, **¿cómo deben cambiar los valores de `inicio` y de `fin`?**

2. Considerando el uso de índices, **¿cómo expresarías la condición del caso base de éxito (cuando la palabra tiene 0 o 1 letras restantes)?** *(Pista: piensa en qué relación tienen `inicio` y `fin` cuando ya no quedan letras por comparar)*.

User prompt: Al valor inicial de "Inicio" le sumo de 1 en 1, mientras que a la de "Fin" se le va restando de 1 en 1. La condición finaliza cuando el inicio sea mayor que el de fin

Response: ¡Brillante! 🌟 Entendiste la lógica de los índices a la perfección.

- **Paso recursivo:** Avance del índice `inicio + 1` y retroceso del índice `fin - 1`.
- **Caso base:** Cuando `inicio >= fin`, significa que ya comparamos todas las letras y cruzamos el centro sin encontrar diferencias. ¡Es un palíndromo! (`true`).

5. Estructurando el código en Java ☕

Ahora que tenemos todas las piezas del rompecabezas, ¡vamos a unir la estructura en código Java!

Recuerda cómo dividimos la responsabilidad:

1. **Método *Wrapper* (Público):** Recibe solo la `String palabra`, prepara los índices iniciales y llama al auxiliar.
2. **Método Auxiliar (Privado y Recursivo):** Recibe la `String palabra`, `int inicio`, `int fin` y contiene la lógica de los casos base y la recursión.

Aquí tienes un esqueleto incompleto del código. **¿Cómo completarías los espacios señalados con `???`?**

```
public class Palindromo {

    // Método Wrapper (Público)
    public static boolean esPalindromo(String palabra) {
        // Llama al método recursivo pasando palabra, inicio (0) y fin (última posición)
        return esPalindromoAuxiliar(palabra, 0, ???);
    }

    // Método Recursivo (Privado)
    private static boolean esPalindromoAuxiliar(String palabra, int inicio, int fin) {
        // Caso Base 1: Si los índices se cruzaron o son iguales
        if (inicio >= fin) {
            return true;
        }

        // Caso Base 2: Si los caracteres en las posiciones actuales NO coinciden
        if (palabra.charAt(inicio) != palabra.charAt(fin)) {
            return ???;
        }

        // Paso Recursivo: Avanzar inicio, retroceder fin
        return esPalindromoAuxiliar(palabra, ???, ???);
    }
}
```

¿Qué valores pondrías en los 4 espacios con `???`?

User prompt: 1. Iría la parte en la que se inicia "fin", tomando la ultima posición de la cadena, 2. Iría la parte que retorna un "false", 3 y 4. Irían la parte de los dos contadores

Response: ¡Excelente trabajo, Jaime! 🥳 Has resuelto correctamente la lógica de cada uno de los puntos:

1. **En la posición 1:** Se pasa la última posición válida de la cadena, que es `palabra.length() - 1`.
2. **En la posición 2:** Retorna `false`, ya que si dos letras de los extremos no coinciden, rompemos la recursión inmediatamente.
3. **En las posiciones 3 y 4:** Se pasa la actualización de los contadores: `inicio + 1` y `fin - 1`.

Código Final Implementado 💻

Así es como queda tu método recursivo completo en Java aplicando el patrón *wrapper*:

```
public class Palindromo {

    // Método Wrapper (Público)
    public static boolean esPalindromo(String palabra) {
        // Inicializa el índice 'fin' en la última posición válida
        return esPalindromoAuxiliar(palabra, 0, palabra.length() - 1);
    }

    // Método Recursivo (Privado)
    private static boolean esPalindromoAuxiliar(String palabra, int inicio, int fin) {
        // Caso Base 1: Los índices se cruzaron o están en la misma letra
        if (inicio >= fin) {
            return true;
        }

        // Caso Base 2: Los caracteres no coinciden
        if (palabra.charAt(inicio) != palabra.charAt(fin)) {
            return false;
        }

        // Paso Recursivo: Acercar los índices hacia el centro
        return esPalindromoAuxiliar(palabra, inicio + 1, fin - 1);
    }
}
```

Recapitulación de lo que aprendiste 🧠

- **Eficiencia:** Usar índices (`inicio` y `fin`) evita crear nuevas cadenas con `substring()`, ahorrando memoria en la pila de llamadas.
- **Patrón Wrapper:** Permite ofrecer una interfaz limpia al usuario (`esPalindromo(palabra)`) mientras ocultas la complejidad de los parámetros recursivos en un método auxiliar privado.
- **Casos Base:** Definir cuándo detener la recursión es esencial para evitar el desbordamiento de pila (*StackOverflowError*).

¡Has diseñado el método tú mismo paso a paso! Para cerrar esta sesión de tutoría, ¿qué te gustaría hacer a continuación?

1. **Casos especiales:** Analizar qué pasa con mayúsculas, espacios o acentos (ejemplo: `"Radar"` o `"anita lava la tina"`).
2. **Complejidad:** Analizar la complejidad en tiempo ($O(n)$) y en espacio ($O(n)$) de esta solución.
3. **Probar otro ejercicio:** Resolver un problema recursivo similar en Java.
